We are passing two variables to the trainer: `dm` and `model`, where `dm` is a variable that represents the `DeltaDataModule` class, and the `model` is assigned to the `LitCVNet` class with the corresponding number of classes we are expecting to train our model on. Note that `trainer.fit(model, dm)` is another way of calling the same thing (we have included this alternative code as a comment). Keeping this structure of your training function helps you to evaluate various strategies while selecting the best parameters, model families, and data loaders for your task.

Now let's move on to the DataLoader. As we explained in *Chapter 3*, we are going to store our images under a Delta Lake format and use a new open-source project called DeltaTorch to load in our images (see *Further Reading* for more details on DeltaTorch).

```
from deltatorch import create_pytorch_dataloader, FieldSpec

class DeltaDataModule(pl.LightningDataModule):
    """
    Creating a Data loading module with Delta Torch loader
    """
    def __init__(self):
        super().__init__()
        self.num_classes = 6
        # Here we are applying the same Transformation
        # in Production case scenario you probably would like to have to separate transformers for your data
        # 1 for Train and another one for test.
        # See the Native Torch example below.

        self.transform = transforms.Compose([
                transforms.Resize((150,150)),
                transforms.RandomHorizontalFlip(p=0.5), # randomly flip and rotate
                transforms.ColorJitter(0.3,0.4,0.4,0.2),
                transforms.ToTensor(),
                transforms.Normalize((0.425, 0.415, 0.405), (0.205, 0.205, 0.205))
                ])

    def dataloader(self, path: str, batch_size=32):

        return create_pytorch_dataloader(
            path,
            id_field="id",
            fields=[
                FieldSpec("content", load_image_using_pil=True, transform=self.transform),
                FieldSpec("label_id"),
            ],
            shuffle=True,
            batch_size=batch_size,
        )

    def train_dataloader(self):
        return self.dataloader(train_delta_path, batch_size=64)

    def val_dataloader(self):
        return self.dataloader(val_delta_path, batch_size=64)

    def test_dataloader(self):
        return self.dataloader(val_delta_path)
```

Figure 6.27 – A class to store our images using DeltaTorch